## CS 696A – Full-Stack Enterprise Application Development – Prof. Radiy Matveev

**Instructions**

- Create a file named "CS696A_WK3_{FirstName}_{LastName}.docx" (e.g., CS696A_WK3_Doosan_Baik.docx).
- Complete three (3) exercises below by answering seven (6) questions inside the DOCX file created.
- Upload the file to the assignment section of the course homepage at https://classes.pace.edu before the class end time.

**Exercise 1: Create your Github Account & Website**

If you already have a Github account, log in and skip to Step 4.

1. Go to https://www.github.com
2. Click 'Sign Up' button at the top right corner.
3. Create your Github account following the steps; remember the username of the account.
4. Create a new repository /***week3***.
5. Click upload file and drag your Lab 1 directory EXCEPT for *node_modules* directory.
6. Go to your repository.

**Question 1:** Take a screenshot of the repository created from the web browser. Take a screenshot and place it within the DOCX file.

7. Add ssh key to your repository.

```
ssh-keygen
cat ~/.ssh/id_rsa.pub
```

8. Copy the content from the above command to your github repository via green CODE dropdown button.
9. **Make sure** the commands above were successful by running

```
git remote -v
```

**IF** you see ***https://*** in your URL run

```
git remote set-url origin git@github.com:USERNAME/REPONAME.git
```

10. Click a green 'Code' button and find a URL under SSH; it is a URL representing your website's git repository.

**Question 2:** Paste the URL within the DOCX file.

11. Read https://pages.github.com/ to understand further about Github Website (Github Pages).

**Exercise 2: Set up Git Client**

If you are using a Macbook, skip to Step 4. MacOS has Git installed by default.

12. Go to https://git-scm.com/download/win and download Windows Installer.
13. Follow the steps inside the installer to complete the installation steps.
14. Search 'Git Bash' on Start Menu, and confirm a command line prompt (Terminal window) shows up.
15. Within the Terminal window (On Windows, Git Bash; on Macbook, Terminal), run a command 'git help -a'.

**Question 3:** Take a screenshot of the output of the command 'git help -a' within the Terminal window. Take a screenshot and place it within the DOCX file.

16. Run the following two (2) command to set up your information within Git; this personal information will be used to keep track of the people who made the latest changes inside the repository.
    **Note**: Replace <YOUR NAME> and <YOUR EMAIL> with your own details.

```
git config --global user.name "<YOUR NAME>"
git config --global user.email "<YOUR EMAIL>"
```

**Exercise 3: Practice Git Workflow**

17. We will clone a git repository corresponding to your Github website (select SSH link); run the following command within the Terminal.
    **Note**: Replace <REPOSITORY URL> with the URL identified for **Question 2** above.

```
Make directory called Lab3
cd Lab3
git clone <REPOSITORY URL>
```

18. Now, you will see all the contents inside the repository are copied down to the new folder created. Run bellow command at the root. You should see *node_modules* directory created at the root.

```
npm install
```

19. Within the Terminal, deploy docker container.

```
docker run -d --name mongo -p 27017:27017 mongo
```

20. Create *.env* file. Paste content from your Lab1 .env file into the newly created (after you create .env file make sure your application is deployable by running ***npm start***)
21. Try testing your test file.
22. Create *.gitignore* file (at this point ask a question what directory should we ignore)
23. We will commit and push the new files now; run the following commands:

    `git status` (**Note**: the command may show fatal error; read the instructions and take actions accordingly).
    `git add .`
    `git commit -m "Add .env and .gitignore"`
    `git push` (**Note**: the command may ask you to log in; read the instructions and take actions).

24. Refresh Github repository on the web browser; confirm '.gitignore' and '.env' exist now; wait 1-2 minutes.

**Question 4:** Go to `repository` URL on the web browser. Take a screenshot of your web browser showing ".gitignore", ".env" and place it inside the DOCX file.

25. Now, we will practice the usage of feature branch. Within the Terminal or via github.com, run the following command to create a new feature branch.

```
git checkout -b add-readme #OR via github.com
```

26. Add README.md file by creating a new file in week3 root and add below content.

```
    This is a REAMEME file for Lab3.
```

27. Now, we will push the change in the feature branch so that a pull request can be created; a pull request is used to review a change made for a particular feature and merge the change into the main branch. Run the following commands:

    `git status` (**Note**: the command may show fatal error; read the instructions and take actions accordingly).
    `git add .`
    `git commit -m 'Add README.md'`
    `git push` (**Note**: the command may show an error; read the instructions and take actions accordingly).

28. Refresh Github repository on the web browser. Click 'main' at the top left corner; now you will see 'add-readme' (a new branch we created) under the menu. Click 'add-readme' branch name.

**Question 5:** Take a screenshot of your Github repository showing 'add-readme' branch name, and place it in the DOCX file.

29. Click 'Contribute' at top right corner, and click a green 'Open Pull Request' button.

**Question 6:** Scroll down and see the change of the file; take a screenshot of the change of the file and place it in the DOCX file.

30. Click green 'Create Pull Request' button.
31. Click green 'Merge Pull Request' button; now, your changes from the feature branch are merged to 'main' branch that is reflected on your Github website. Wait 2-3 minutes.

32. Within the Terminal, run the following command to go back to 'main' branch.

```
git checkout main
```

33. Notice that the new contents added doesn't exist.
34. Run the following command to pull the changes that were merged in step 12.

```
git pull
```